

# 1 Positive rational magnitudes as barcodes

Source: PrimeTensor/MulRat.lean

## What this file establishes

PrimeTensor/Ratio.lean introduced PrimeRatio, an oriented pair of prime multisets, and defined the cross-product relation Same on it — but proved nothing whatever about that relation. This file supplies everything that was missing and then takes the quotient.

The result, MulRat, is a commutative group: multiplication is associative and commutative with unit 1, and every element has an inverse. All six laws are equalities of elements, not statements modulo a relation.

One step in the file does real work. Reflexivity and symmetry of Same are immediate, but transitivity requires cancelling a common factor, and cancellation on  $\mathbb{N}$  is valid only against a nonzero factor. The factor available is the evaluation of a prime multiset, and it is nonzero because no bag evaluates below 1. This is the point at which the project's refusal of a zeroth stage, decided in PrimeTensor/Depth.lean and carried into PrimeTensor/Prime.lean, is actually spent.

Finally, and as the module header states, no value of type  $\mathbb{Q}$  or  $\mathbb{R}$  occurs anywhere in the file. Magnitudes are barcodes throughout;  $\mathbb{N}$  enters only through eval, and only to compare.

## 1.1 Positivity

**Lemma 1.1** (PrimeRatio.multiset\_eval\_pos).  $0 < \text{eval}(q)$  for every  $q : \text{PrimeMultiset}$ .

```
private theorem multiset_eval_pos (q : PrimeMultiset) : 0 < q.eval := by
  refine Quotient.inductionOn q ?_
  intro b
  exact lt_of_lt_of_le Nat.zero_lt_one (PrimeBag.one_le_eval b)
```

*Proof.* By Quotient.inductionOn it suffices to treat a class represented by a bag  $b$ , where eval reduces to PrimeBag.eval( $b$ ). Then  $0 < 1$  and  $1 \leq \text{PrimeBag.eval}(b)$  — the latter is one\_le\_eval from PrimeTensor/Prime.lean — compose to  $0 < \text{PrimeBag.eval}(b)$ .  $\square$

The lemma is private, hence visible only inside this file. That is appropriate: it exists to feed the single cancellation step below, and the public statement of that fact is one\_le\_eval, which is sharper.

## 1.2 Cross-product equality is an equivalence

**Lemma 1.2** (same\_refl and same\_symm). Same is reflexive and symmetric.

*Proof.* Unfolding the definition from PrimeTensor/Ratio.lean, Same( $a, a$ ) is  $\text{eval}(a.\text{upper}) \cdot \text{eval}(a.\text{lower}) = \text{eval}(a.\text{upper}) \cdot \text{eval}(a.\text{lower})$ , closed by rfl; and Same( $a, b$ ) and Same( $b, a$ ) are the two sides of one equation of naturals, so symmetry is Eq.symm. The two carry @[refl] and @[symm].  $\square$

**Lemma 1.3** (Transitivity, PrimeRatio.same\_trans). If Same( $a, b$ ) and Same( $b, c$ ) then Same( $a, c$ ).

```

@[trans] theorem same_trans {a b c : PrimeRatio}
  (hab : Same a b) (hbc : Same b c) : Same a c := by
  unfold Same at hab hbc ⊢
  apply Nat.eq_of_mul_eq_mul_right (multiset_eval_pos b.lower)
  calc
    (a.upper.eval * c.lower.eval) * b.lower.eval
      = (a.upper.eval * b.lower.eval) * c.lower.eval := by ac_rfl
    _ = (b.upper.eval * a.lower.eval) * c.lower.eval := by rw [hab]
    _ = (b.upper.eval * c.lower.eval) * a.lower.eval := by ac_rfl
    _ = (c.upper.eval * b.lower.eval) * a.lower.eval := by rw [hbc]
    _ = (c.upper.eval * a.lower.eval) * b.lower.eval := by ac_rfl

```

*Proof.* Write  $u_x = \text{eval}(x.\text{upper})$  and  $l_x = \text{eval}(x.\text{lower})$ . The hypotheses are  $u_a l_b = u_b l_a$  and  $u_b l_c = u_c l_b$ , and the goal is  $u_a l_c = u_c l_a$ .

The obvious route — divide the two hypotheses — is unavailable, since  $\mathbb{N}$  has no division. Instead one multiplies the goal by  $l_b$  and cancels it afterwards. By `Nat.eq_of_mul_eq_mul_right`, applied with the positivity of  $l_b$  supplied by Lemma 1.1, it is enough to prove

$$(u_a l_c) l_b = (u_c l_a) l_b,$$

and that is the displayed chain: reassociate to expose  $u_a l_b$ , rewrite it to  $u_b l_a$  by the first hypothesis, reassociate to expose  $u_b l_c$ , rewrite it to  $u_c l_b$  by the second, and reassociate once more. Each reassociation step is closed by `ac_rfl`, which decides equalities that hold by associativity and commutativity of multiplication alone.  $\square$

*Design note 1.4* (Where the missing zero is spent). Cancellation is the whole difficulty, and cancellation on  $\mathbb{N}$  is valid only against a factor that is nonzero. The factor here is  $l_b$ , the evaluation of a prime multiset, and Lemma 1.1 makes it positive by tracing back through `one_le_eval` to the fact that a prime bag is a product of atoms each greater than 1, terminated by the pivot one rather than by a zero.

Had the carrier admitted a zeroth stage, this step would have needed a side condition, and the side condition would have propagated into setoid, into every lift out of the quotient, and into every downstream statement about `MulRat`. Instead there is nothing to discharge: the representation cannot express the bad case.

**Definition 1.5** (The setoid, `PrimeTensor.PrimeRatio.setoid`). `setoid : Setoid(PrimeRatio)` is `Same` with the equivalence proof assembled from Lemmas 1.2 and 1.3.

### 1.3 Congruence and the laws at representative level

**Lemma 1.6** (`mul_same` and `inv_same`). *Same is respected by multiplication and by inversion: if  $\text{Same}(a_1, a_2)$  and  $\text{Same}(b_1, b_2)$  then  $\text{Same}(a_1 b_1, a_2 b_2)$ ; and if  $\text{Same}(a, b)$  then  $\text{Same}(a^{-1}, b^{-1})$ .*

*Proof.* For the first, `upper_mul` and `lower_mul` from `PrimeTensor/Ratio.lean` turn the goal into

$$(u_{a_1} u_{b_1})(l_{a_2} l_{b_2}) = (u_{a_2} u_{b_2})(l_{a_1} l_{b_1}),$$

and regrouping gives  $(u_{a_1} l_{a_2})(u_{b_1} l_{b_2})$ , which the two hypotheses rewrite to  $(u_{a_2} l_{a_1})(u_{b_2} l_{b_1})$ , which regroups to the right-hand side. For the second, `inv_upper` and `inv_lower` turn the goal into  $l_a u_b = l_b u_a$ , and the hypothesis  $u_a l_b = u_b l_a$  gives it after commuting each side. Both proofs are chains of `ac_rfl` steps around a single rewrite.  $\square$

**Lemma 1.7** (`one_upper` and `one_lower`).  $(1 : \text{PrimeRatio}).\text{upper} = 1$  and  $(1 : \text{PrimeRatio}).\text{lower} = 1$ .

```
@[simp] theorem one_upper : (1 : PrimeRatio).upper = 1 := rfl
@[simp] theorem one_lower : (1 : PrimeRatio).lower = 1 := rfl
```

*Proof.* `rfl`: the unit of `PrimeRatio` is the literal  $\langle 1, 1 \rangle$ , and projecting a literal reduces.  $\square$

**Lemma 1.8** (The laws, up to `Same`). *For all  $a, b, c : \text{PrimeRatio}$ :*

|                                   |                               |
|-----------------------------------|-------------------------------|
| $\text{Same}(1 \cdot a, a),$      | <code>one_mul_same</code>     |
| $\text{Same}(a \cdot 1, a),$      | <code>mul_one_same</code>     |
| $\text{Same}((ab)c, a(bc)),$      | <code>mul_assoc_same</code>   |
| $\text{Same}(ab, ba),$            | <code>mul_comm_same</code>    |
| $\text{Same}((a^{-1})^{-1}, a),$  | <code>inv_inv_same</code>     |
| $\text{Same}(a \cdot a^{-1}, 1),$ | <code>mul_inv_same_one</code> |

*Proof.* Each unfolds `Same`, rewrites the components with the evaluation lemmas of `PrimeTensor/Ratio.lean` — `upper_mul`, `lower_mul`, `inv_upper`, `inv_lower` — together with Lemma 1.7 and `PrimeMultiset.eval_one`, and closes the resulting identity of naturals. The unit laws close by the unit laws of  $\mathbb{N}$ ; associativity and commutativity close by `ac_rfl`; double inversion closes by simplification alone, both sides becoming literally the same expression once the two swaps cancel; and `Same( $aa^{-1}$ , 1)` reduces to  $u_a l_a = l_a u_a$ , which is `Nat.mul_comm`.  $\square$

Note what these say and do not say. `PrimeRatio` itself does not satisfy any of the six laws —  $a \cdot 1$  and  $a$  are different terms, and  $a \cdot a^{-1}$  is  $\langle u_a l_a, l_a u_a \rangle$ , not  $\langle 1, 1 \rangle$ . Each law holds only after `Same` has been applied, which is exactly the input the quotient needs.

## 1.4 The quotient

**Definition 1.9** (`PrimeTensor.MulRat`). `MulRat`  $\equiv$  `Quotient(PrimeRatio.setoid)`, an `abbrev` and hence reducible. `ofRatio( $q$ )` is the class of  $q$ , and `one`  $\equiv$  `ofRatio(1)`.

**Definition 1.10** (Induced operations). The declarations `MulRat.mul` and `MulRat.inv` lift multiplication and inversion through the quotient; `One`, `Mul` and `Inv` instances give them their notation.

```
def mul (a b : MulRat) : MulRat :=
  Quotient.liftOn2 a b
    (fun x y => ofRatio (x * y))
  (by
    intro a1 b1 a2 b2 ha hb
    change PrimeRatio.Same a1 a2 at ha
    change PrimeRatio.Same b1 b2 at hb
    change Quotient.mk PrimeRatio.setoid (a1 * b1) =
      Quotient.mk PrimeRatio.setoid (a2 * b2)
    apply Quotient.sound
    exact PrimeRatio.mul_same ha hb)
```

Each lift carries a well-definedness obligation, and each is exactly one of the congruence lemmas: `mul_same` for the binary lift, `inv_same` for the unary one, in both cases followed by `Quotient.sound` to convert the resulting `Same` into an equality of classes.

**Theorem 1.11** (MulRat is a commutative group). *For all  $a, b, c : \text{MulRat}$ ,*

$$1 \cdot a = a, \quad a \cdot 1 = a, \quad (ab)c = a(bc), \quad ab = ba, \quad (a^{-1})^{-1} = a, \quad a \cdot a^{-1} = 1,$$

*these being the declarations `one_mul`, `mul_one`, `mul_assoc`, `mul_comm`, `inv_inv` and `mul_inv`.*

*Proof.* Every one has the same three-step shape. By `Quotient.inductionOn` (or its two- and three-variable forms) it suffices to prove the statement for classes of representatives; there the goal becomes an equality of two classes; `Quotient.sound` reduces that to the corresponding `Same` statement, which is the matching item of Lemma 1.8.  $\square$

*Remark 1.12* (What is registered, and what is not). The four laws that make `MulRat` a group are theorems, but no `Group` or `CommGroup` instance is declared — only `One`, `Mul` and `Inv`. So Mathlib’s group lemmas are not available for `MulRat` downstream, and each fact must be taken from this file by name. Note also that  $a^{-1} \cdot a = 1$  is not stated, although it follows from `mul_inv` and `mul_comm`; and that `one_mul`, `mul_one`, `inv_inv` and `mul_inv` carry `@[simp]` while `mul_assoc` and `mul_comm` do not, as is usual for laws that would loop the simplifier.

*Remark 1.13* (What the quotient is). Sending  $a$  to `eval(a.upper)/eval(a.lower)` maps `PrimeRatio` onto  $\mathbb{Q}_{>0}$ : it is surjective because every positive rational is a quotient of two positive integers and every positive integer is a product of primes, and by the cross-product characterisation in `PrimeTensor/Ratio.lean` two ratios have the same image precisely when `Same` relates them. So `MulRat` is isomorphic to  $(\mathbb{Q}_{>0}, \cdot, 1)$  as a group, and Theorem 1.11 is the barcode-level reflection of that. The isomorphism is stated here for the reader; the file constructs no such map, and consistently with its header no value of type  $\mathbb{Q}$  or  $\mathbb{R}$  appears in it.

*Remark 1.14* (The order does not descend here). `PrimeTensor/Ratio.lean` also defined `Lt` and `Within`. This file proves nothing about either, and `MulRat` carries no order: there is no `LT MulRat` instance and no lemma showing `Lt` respects `Same`, though it does. Only the multiplicative structure is transported across the quotient.

## Summary of the correspondence

| Lean declaration                          | Kind        | Here      |
|-------------------------------------------|-------------|-----------|
| <code>PrimeRatio.multiset_eval_pos</code> | private thm | Lem. 1.1  |
| <code>PrimeRatio.same_refl</code>         | refl thm    | Lem. 1.2  |
| <code>PrimeRatio.same_symm</code>         | symm thm    | Lem. 1.2  |
| <code>PrimeRatio.same_trans</code>        | trans thm   | Lem. 1.3  |
| <code>PrimeRatio.setoid</code>            | definition  | Def. 1.5  |
| <code>PrimeRatio.mul_same</code>          | theorem     | Lem. 1.6  |
| <code>PrimeRatio.inv_same</code>          | theorem     | Lem. 1.6  |
| <code>PrimeRatio.one_upper</code>         | simp thm    | Lem. 1.7  |
| <code>PrimeRatio.one_lower</code>         | simp thm    | Lem. 1.7  |
| <code>PrimeRatio.one_mul_same</code>      | theorem     | Lem. 1.8  |
| <code>PrimeRatio.mul_one_same</code>      | theorem     | Lem. 1.8  |
| <code>PrimeRatio.mul_assoc_same</code>    | theorem     | Lem. 1.8  |
| <code>PrimeRatio.mul_comm_same</code>     | theorem     | Lem. 1.8  |
| <code>PrimeRatio.inv_inv_same</code>      | theorem     | Lem. 1.8  |
| <code>PrimeRatio.mul_inv_same_one</code>  | theorem     | Lem. 1.8  |
| <code>PrimeTensor.MulRat</code>           | abbrev      | Def. 1.9  |
| <code>MulRat.ofRatio</code>               | definition  | Def. 1.9  |
| <code>MulRat.one</code>                   | definition  | Def. 1.9  |
| <code>MulRat.mul</code>                   | definition  | Def. 1.10 |
| <code>MulRat.inv</code>                   | definition  | Def. 1.10 |
| <code>One, Mul, Inv</code>                | instances   | Def. 1.10 |
| <code>MulRat.one_mul</code>               | simp thm    | Thm. 1.11 |
| <code>MulRat.mul_one</code>               | simp thm    | Thm. 1.11 |
| <code>MulRat.mul_assoc</code>             | theorem     | Thm. 1.11 |
| <code>MulRat.mul_comm</code>              | theorem     | Thm. 1.11 |
| <code>MulRat.inv_inv</code>               | simp thm    | Thm. 1.11 |
| <code>MulRat.mul_inv</code>               | simp thm    | Thm. 1.11 |

Every declaration of `PrimeTensor/MulRat.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. Remarks 1.13 and 1.14 are stated for the reader and are not declarations of the file.